

# Diabetes Analysis MLP

## Data Preprocessing

```
library(dplyr)
```

```
##  
## Attaching package: 'dplyr'  
  
## The following objects are masked from 'package:stats':  
##  
##   filter, lag  
  
## The following objects are masked from 'package:base':  
##  
##   intersect, setdiff, setequal, union
```

```
library(tidyr)
```

```
# Load data  
df <- read.csv("data/diabetic_data.csv", stringsAsFactors = FALSE, na.strings = c("NA", ""))  
ids <- read.csv("data/IDS_mapping.csv", stringsAsFactors = FALSE)  
  
cat("Raw data dimensions:", nrow(df), "x", ncol(df), "\n")
```

```
## Raw data dimensions: 101766 x 50
```

```
# Replace ? with NA  
df[df == "?"] <- NA  
  
# Drop useless columns  
# Weight is dropped due to a large number of missing values  
df <- df %>% select(-weight)  
df <- df %>% select(-payer_code)  
df <- df %>% select(-medical_specialty)  
df <- df %>% select(-examide)  
df <- df %>% select(-citoglipton)  
df <- df %>% select(-encounter_id)  
df <- df %>% select(-patient_nbr)  
  
# Binarize target variable  
df$readmitted <- as.integer(df$readmitted == "<30")  
  
# Encode age
```

```

age_map <- c(
  "[0-10)" = 0, "[10-20)" = 1, "[20-30)" = 2, "[30-40)" = 3, "[40-50)" = 4, "[50-60)" = 5,
  "[60-70)" = 6, "[70-80)" = 7, "[80-90)" = 8, "[90-100)" = 9
)
# Iteratively apply map to df
df$age <- age_map[df$age]

# Create binary "was test taken" features for max_glu_serum and A1Cresult, as they are largely missing
df$A1C_taken <- as.integer(!is.na(df$A1Cresult))
df$glu_taken <- as.integer(!is.na(df$max_glu_serum))

df <- df %>% select(-A1Cresult, -max_glu_serum)

# Group diagnosis codes
# Immediately one-hot encoding them would result in too many columns that are 0 for most patients
group_icd9 <- function(code) {
  if (is.na(code)) return("Other")

  num <- suppressWarnings(as.numeric(code))
  if (is.na(num)) return("Other")

  # Code categories retrieved from source [1] in final report
  # https://onlinelibrary.wiley.com/doi/10.1155/2014/781670
  if (num >= 390 & num <= 459 | num == 785) return("Circulatory")
  if (num >= 460 & num <= 519 | num == 786) return("Respiratory")
  if (num >= 520 & num <= 579 | num == 787) return("Digestive")
  if (num >= 250 & num < 251) return("Diabetes") # Has potential decimal values
  if (num >= 800 & num <= 999) return("Injury")
  if (num >= 710 & num <= 739) return("Musculoskeletal")
  if (num >= 580 & num <= 629 | num == 788) return("Genitourinary")
  if (num >= 140 & num <= 239) return("Neoplasms")
  return("Other")
}

# Apply grouping function to all diagnoses columns
df$diag_1 <- sapply(df$diag_1, group_icd9)
df$diag_2 <- sapply(df$diag_2, group_icd9)
df$diag_3 <- sapply(df$diag_3, group_icd9)

# Encode medicine columns
# In dataset, these columns are represented as values "No", "Steady", "Up", or "Down".
# Simplify these to binary, prescribed or not.
medicine_cols <- c(
  "metformin", "repaglinide", "nateglinide", "chlorpropamide", "glimepiride", "acetoexamide",
  "glipizide", "glyburide", "tolbutamide", "pioglitazone", "rosiglitazone", "acarbose", "miglitol",
  "troglitazone", "tolazamide", "insulin", "glyburide.metformin", "glipizide.metformin",
  "glimepiride.pioglitazone", "metformin.rosiglitazone", "metformin.pioglitazone"
)

for (col in medicine_cols) {
  if (col %in% colnames(df)) {

```

```

    df[[col]] <- as.integer(df[[col]] != "No")
  }
}

# Decode id columns using IDS_mapping.csv
# admission_type_id, discharge_disposition_id, admission_source_id

# Group discharge disposition ids
simplify_discharge <- function(id) {
  id <- as.integer(id)
  if (is.na(id)) return("Unknown")
  if (id == 1) return("Home")
  if (id %in% c(2,3,4,5)) return("Transfer")
  if (id == 6) return("Home_Health")
  if (id == 7) return("AMA")
  if (id %in% c(11,19,20,21)) return("Expired")
  if (id %in% c(13,14)) return("Hospice")
  return("Other")
}

# Admission type ids
simplify_admission <- function(id) {
  id <- as.integer(id)
  if (is.na(id)) return("Unknown")
  if (id == 1) return("Emergency")
  if (id == 2) return("Urgent")
  if (id == 3) return("Elective")
  if (id == 4) return("Newborn")
  return("Other")
}

# Group admission source ids
simplify_source <- function(id) {
  id <- as.integer(id)
  if (is.na(id)) return("Unknown")
  if (id == 7) return("EmergencyRoom")
  if (id == 1) return("PhysicianReferral")
  if (id %in% c(2,3)) return("ClinicOrHMO")
  if (id %in% c(4,5,6,10)) return("Transfer")
  return("Other")
}

# Apply simplification/grouping functions to all relevant columns
df$discharge_disposition_id <- sapply(df$discharge_disposition_id, simplify_discharge)
df$admission_type_id <- sapply(df$admission_type_id, simplify_admission)
df$admission_source_id <- sapply(df$admission_source_id, simplify_source)

# Binary encode gender
df <- df %>% filter(gender != "Unknown/Invalid" | is.na(gender))
df$gender <- as.integer(df$gender == "Male")

# Binary encode whether medicine was changed during the encounter
df$change <- as.integer(df$change == "Ch")

```

```

# Binary encode diabetes medication
df$diabetesMed <- as.integer(df$diabetesMed == "Yes")

# One-hot encoding of categoricals
one_hot <- function(df, col) {
  vals <- sort(unique(df[[col]]))
  vals <- vals[-1]
  for (v in vals) {
    new_col <- paste0(col, "_", v)
    df[[new_col]] <- as.integer(df[[col]] == v)
  }
  df[[col]] <- NULL
  return(df)
}

df <- one_hot(df, 'race')
df <- one_hot(df, 'diag_1')
df <- one_hot(df, 'diag_2')
df <- one_hot(df, 'diag_3')
df <- one_hot(df, 'admission_type_id')
df <- one_hot(df, 'discharge_disposition_id')
df <- one_hot(df, 'admission_source_id')

# Drop rows that still have NAs after preprocessing
n_before <- nrow(df)
df <- df %>% drop_na()
cat("Dropped", n_before - nrow(df), "rows")

## Dropped 2271 rows

# Train test split - 80/20
# Stratified - split done separately for positive and negative rows
set.seed(42)

pos_idx <- which(df$readmitted == 1)
neg_idx <- which(df$readmitted == 0)

train_pos <- sample(pos_idx, size = floor(0.8 * length(pos_idx)))
train_neg <- sample(neg_idx, size = floor(0.8 * length(neg_idx)))

train_idx <- c(train_pos, train_neg)
test_idx <- setdiff(seq_len(nrow(df)), train_idx)

train_df <- df[train_idx, ]
test_df <- df[test_idx, ]

# Scale numeric columns using training set stats
numeric_cols <- c(
  "time_in_hospital", "num_lab_procedures", "num_procedures", "num_medications", "number_outpatient",
  "number_emergency", "number_inpatient", "number_diagnoses", "age"
)

```

```

scale_means <- sapply(train_df[, numeric_cols], mean)
scale_sds <- sapply(train_df[, numeric_cols], sd)

for (col in numeric_cols) {
  train_df[[col]] <- (train_df[[col]] - scale_means[col]) / scale_sds[col]
  test_df[[col]] <- (test_df[[col]] - scale_means[col]) / scale_sds[col]
}

# Separate into x and y
X_train <- as.matrix(train_df %>% select(-readmitted))
y_train <- train_df$readmitted

X_test <- as.matrix(test_df %>% select(-readmitted))
y_test <- test_df$readmitted

```

## MLP

```

# Balance training set by undersampling
set.seed(42)
pos_idx_train <- which(y_train == 1)
neg_idx_train <- which(y_train == 0)
neg_idx_sampled <- sample(neg_idx_train, size = length(pos_idx_train))
balanced_idx <- sample(c(pos_idx_train, neg_idx_sampled))
X_train_bal <- X_train[balanced_idx, ]
y_train_bal <- y_train[balanced_idx]

cat("Balanced train set:", nrow(X_train_bal), "rows, Positive rate:",
    round(mean(y_train_bal) * 100, 2), "%\n")

```

```
## Balanced train set: 17870 rows, Positive rate: 50 %
```

```

# Activation functions
relu <- function(x) ifelse(x > 0, x, 0)
relu_deriv <- function(x) ifelse(x > 0, 1.0, 0)
sigmoid <- function(x) 1.0 / (1.0 + exp(-pmax(-500, pmin(500, x))))

bce_loss <- function(y_hat, y) {
  -mean(y * log(y_hat + 1e-8) + (1 - y) * log(1 - y_hat + 1e-8))
}

# Initialize weights
set.seed(42)
n_input <- ncol(X_train_bal)
q1 <- 64 # nodes in hidden layer 1
q2 <- 32 # nodes in hidden layer 2
n_output <- 1 # output nodes

W1 <- matrix(rnorm(q1 * n_input), nrow = q1) # weights connecting input to hidden layer 1
W2 <- matrix(rnorm(q2 * q1), nrow = q2) # weights connecting hidden layer 1 to hidden layer 2
W3 <- matrix(rnorm(n_output * q2), nrow = n_output) # weights connecting hidden layer 2 to output

```

```

b1 <- matrix(0, nrow = q1, ncol = 1) # bias vector for hidden layer 1
b2 <- matrix(0, nrow = q2, ncol = 1) # bias vector for hidden layer 2
b3 <- matrix(0, nrow = n_output, ncol = 1) # bias vector for output layer

# Define the prediction function
f <- function(X) {
  h1 <- relu(W1 %*% t(X) + matrix(b1, nrow = q1, ncol = nrow(X)))
  h2 <- relu(W2 %*% h1 + matrix(b2, nrow = q2, ncol = nrow(X)))
  return(as.vector(sigmoid(W3 %*% h2 + matrix(b3, nrow = n_output, ncol = nrow(X)))))
}

# Gradient descent
eta <- 0.01
epochs <- 400
n <- nrow(X_train_bal)
errors <- numeric()

X_batch <- t(X_train_bal)
y_batch <- y_train_bal
m <- ncol(X_batch)

for (epoch in 1:epochs) {
  # Forward pass
  z1_pre <- W1 %*% X_batch + matrix(b1, nrow = q1, ncol = m)
  h1 <- relu(z1_pre)

  z2_pre <- W2 %*% h1 + matrix(b2, nrow = q2, ncol = m)
  h2 <- relu(z2_pre)

  z3 <- W3 %*% h2 + matrix(b3, nrow = n_output, ncol = m)
  y_hat <- sigmoid(z3)

  # Backward
  dz3 <- y_hat - matrix(y_batch, nrow = 1)
  dW3 <- (1/m) * dz3 %*% t(h2)
  db3 <- matrix((1/m) * rowSums(dz3))

  mat2 <- t(W3) %*% dz3
  dz2_pre <- mat2 * relu_deriv(z2_pre)
  dW2 <- (1/m) * dz2_pre %*% t(h1)
  db2 <- matrix((1/m) * rowSums(dz2_pre))

  mat1 <- t(W2) %*% dz2_pre
  dz1_pre <- mat1 * relu_deriv(z1_pre)
  dW1 <- (1/m) * dz1_pre %*% t(X_batch)
  db1 <- matrix((1/m) * rowSums(dz1_pre))

  # Gradient descent updates
  W1 <- W1 - eta * dW1
  b1 <- b1 - eta * db1
  W2 <- W2 - eta * dW2
  b2 <- b2 - eta * db2
  W3 <- W3 - eta * dW3

```

```

b3 <- b3 - eta * db3

# Calculate loss
e <- bce_loss(f(X_train_bal), y_train_bal)
errors <- c(errors, e)

if (epoch %% 10 == 0) cat(sprintf("Epoch %d: Loss: %.4f\n", epoch, e))
}

```

```

## Epoch 10: Loss: 6.6557
## Epoch 20: Loss: 6.0506
## Epoch 30: Loss: 5.5678
## Epoch 40: Loss: 5.1260
## Epoch 50: Loss: 4.7143
## Epoch 60: Loss: 4.3253
## Epoch 70: Loss: 3.9565
## Epoch 80: Loss: 3.6061
## Epoch 90: Loss: 3.2737
## Epoch 100: Loss: 2.9616
## Epoch 110: Loss: 2.8647
## Epoch 120: Loss: 4.1672
## Epoch 130: Loss: 3.9539
## Epoch 140: Loss: 3.7940
## Epoch 150: Loss: 3.6473
## Epoch 160: Loss: 3.5103
## Epoch 170: Loss: 3.3796
## Epoch 180: Loss: 3.2558
## Epoch 190: Loss: 3.1386
## Epoch 200: Loss: 3.0275
## Epoch 210: Loss: 2.9219
## Epoch 220: Loss: 2.8216
## Epoch 230: Loss: 2.7267
## Epoch 240: Loss: 2.6366
## Epoch 250: Loss: 2.5511
## Epoch 260: Loss: 2.4698
## Epoch 270: Loss: 2.3926
## Epoch 280: Loss: 2.3191
## Epoch 290: Loss: 2.2494
## Epoch 300: Loss: 2.1832
## Epoch 310: Loss: 2.1203
## Epoch 320: Loss: 2.0605
## Epoch 330: Loss: 2.0038
## Epoch 340: Loss: 1.9499
## Epoch 350: Loss: 1.8986
## Epoch 360: Loss: 1.8499
## Epoch 370: Loss: 1.8036
## Epoch 380: Loss: 1.7595
## Epoch 390: Loss: 1.7176
## Epoch 400: Loss: 1.6777

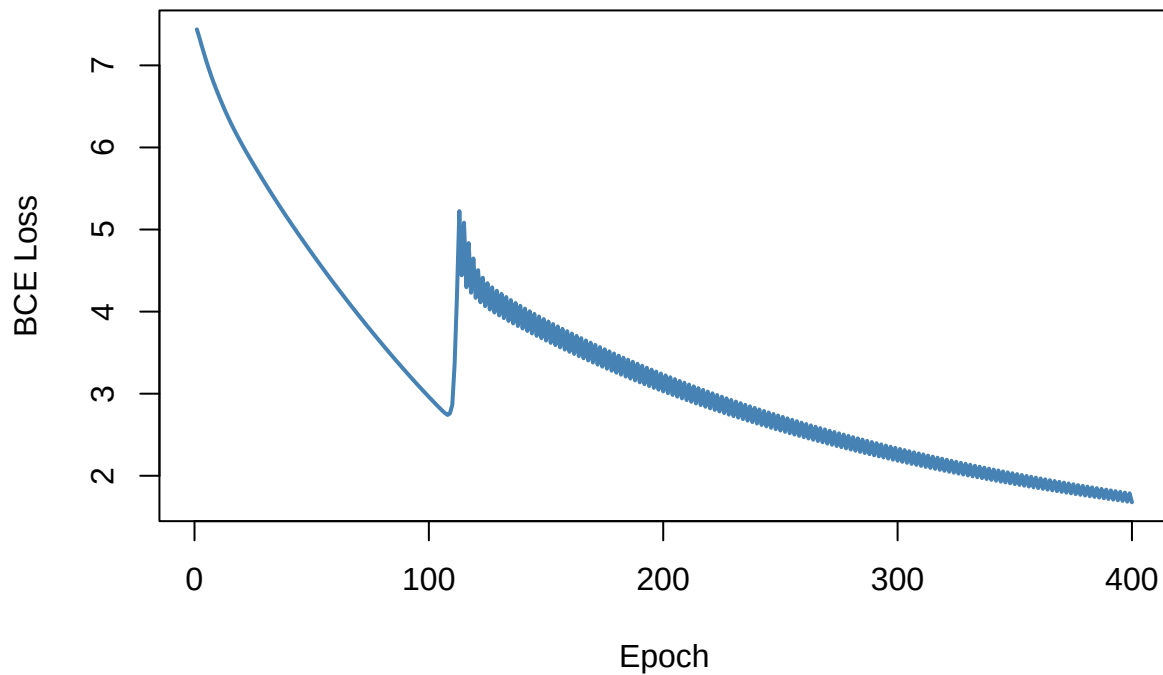
```

```

plot(1:epochs, errors, type = "l", main = "Training Loss over Epochs",
     xlab = "Epoch", ylab = "BCE Loss", col = "steelblue", lwd = 2)

```

## Training Loss over Epochs



```
dev.copy(png, "mlp_loss_curve.png", width = 800, height = 600)
```

```
## quartz_off_screen
##                               3
```

```
dev.off()
```

```
## pdf
##    2
```

```
# Evaluation
y_prob <- f(X_test)

y_pred <- as.integer(y_prob >= 0.1)

conf_matrix <- matrix(0, nrow = 2, ncol = 2,
                      dimnames = list(Predicted = c("0", "1"),
                                       Actual    = c("0", "1")))
for (p in c("0", "1")) for (a in c("0", "1")) {
  conf_matrix[p, a] <- sum(y_pred == as.integer(p) & y_test == as.integer(a))
}
print(conf_matrix)
```

```
##           Actual
```

```
## Predicted      0      1
##              0 12622 1337
##              1  5043   897
```

```
accuracy <- mean(y_pred == y_test)
tp <- conf_matrix["1","1"]
fp <- conf_matrix["1","0"]
fn <- conf_matrix["0","1"]
precision <- if ((tp+fp)>0) tp/(tp+fp) else 0
recall <- if ((tp+fn)>0) tp/(tp+fn) else 0
f1 <- if ((precision+recall)>0) 2*precision*recall/(precision+recall) else 0

cat(sprintf("Accuracy: %.4f\n", accuracy))
```

```
## Accuracy: 0.6794
```

```
cat(sprintf("Precision: %.4f\n", precision))
```

```
## Precision: 0.1510
```

```
cat(sprintf("Recall: %.4f\n", recall))
```

```
## Recall: 0.4015
```

```
cat(sprintf("F1 Score: %.4f\n", f1))
```

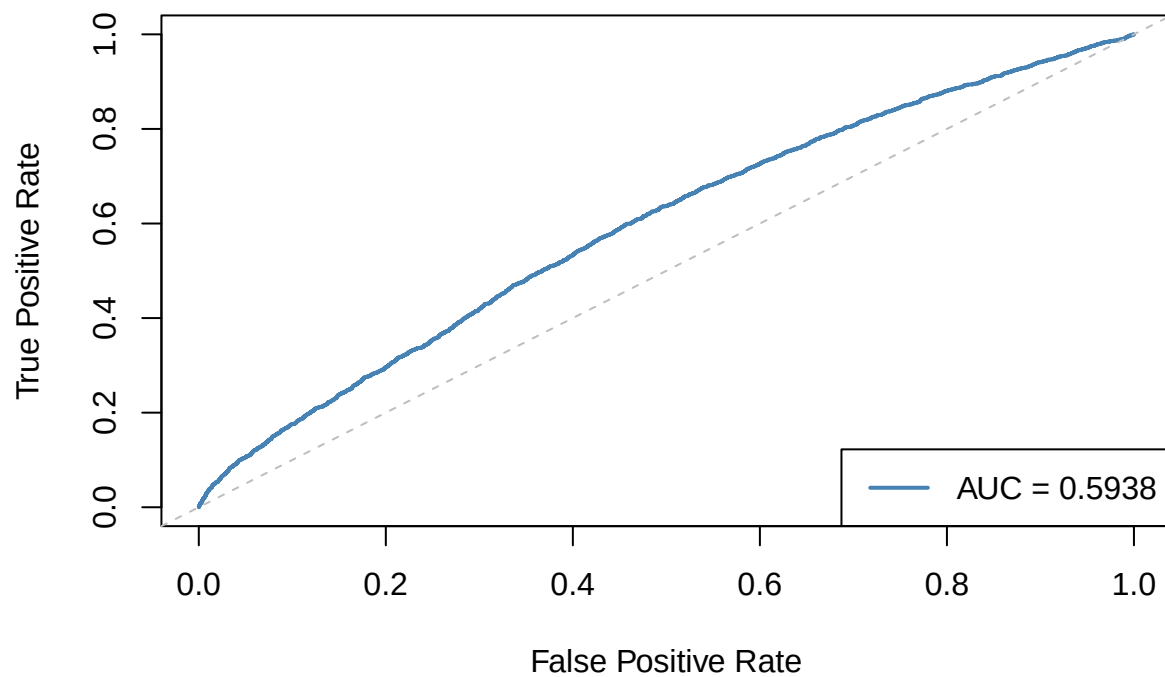
```
## F1 Score: 0.2195
```

```
ord <- order(y_prob, decreasing = TRUE)
y_sorted <- y_test[ord]
n_pos <- sum(y_test == 1)
n_neg <- sum(y_test == 0)
tpr <- cumsum(y_sorted == 1) / n_pos
fpr <- cumsum(y_sorted == 0) / n_neg
auc <- sum(abs(diff(fpr)) * (tpr[-1] + tpr[-length(tpr)])) / 2)
cat(sprintf("AUC-ROC: %.4f\n", auc))
```

```
## AUC-ROC: 0.5938
```

```
plot(fpr, tpr, type = "l", col = "steelblue", lwd = 2,
     main = "ROC Curve - MLP",
     xlab = "False Positive Rate", ylab = "True Positive Rate")
abline(a = 0, b = 1, lty = 2, col = "gray")
legend("bottomright", legend = sprintf("AUC = %.4f", auc),
     col = "steelblue", lwd = 2)
```

## ROC Curve – MLP



```
dev.copy(png, "mlp_roc_curve.png", width = 800, height = 600)
```

```
## quartz_off_screen  
## 3
```

```
dev.off()
```

```
## pdf  
## 2
```